

Your First CRUD App, Explained

A beginner's guide to the student manager you built with PHP, MySQL and Bootstrap

What's inside

- 1 What is CRUD?
- 2 The three parts that work together
- 3 The database
- 4 The files, and how a page is built
- 5 dbconnection.php: talking to the database
- 6 header.php and footer.php
- 7 Read: showing the students
- 8 Create: adding a student
- 9 Messages after each action
- 10 Update: changing a student
- 11 Delete: removing a student
- 12 GET vs POST
- 13 The errors you hit, and what they teach
- 14 Next steps
- 15 Glossary

How to read this guide

Read it in order. Each section builds on the one before. Keep your project open in VS Code next to this PDF and find each piece of code as you read about it.

Code in the grey boxes is copied from your own files.

1. What is CRUD?

CRUD is a name for the four things almost every app does with data. Instagram, a school system and a shopping site all do these four things. Your app does them with a list of students.

Letter	Means	In your app	SQL command
C	Create	Add a new student	INSERT
R	Read	Show all students in the table	SELECT
U	Update	Change a student's name or age	UPDATE
D	Delete	Remove a student	DELETE

Think of it like a notebook

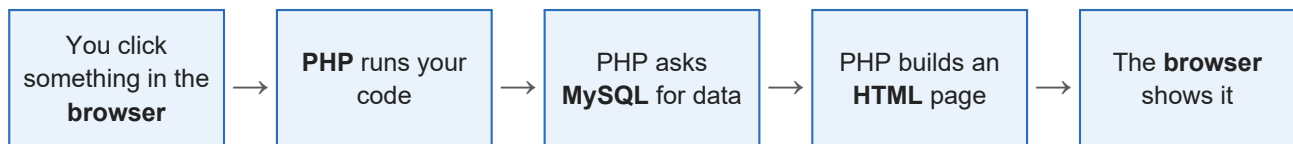
A teacher keeps a notebook with a list of students. They can **write** a new name (create), **look** at the list (read), **cross out and fix** a name (update), or **erase** a name (delete). Your app does exactly the same, except the notebook is a **database**.

2. The three parts that work together

Your app isn't one program. It's three programs talking to each other:

Part	What it does	Analogy
Browser (Chrome, Edge)	Shows the page. You click buttons and type in forms here.	The customer in a restaurant
Apache + PHP (the server)	Runs your <code>.php</code> files and builds the HTML page.	The waiter and the kitchen
MySQL (the database)	Stores the students permanently.	The fridge where food is kept

Every time you open a page or click a button, this happens:



Important: PHP runs before the page reaches you

PHP runs on the **server**, then sends only the finished HTML to the browser. The browser never sees your PHP code. If you right-click the page and choose **View page source**, you'll see HTML but no `<?php ... ?>`.

This is why a PHP file can't be opened by double-clicking it. It has to go through Apache, at an address like `http://localhost/crud_app/`.

XAMPP is a package that installs Apache, PHP and MySQL together. **localhost** means "this computer". The server is running on your own machine.

3. The database

A **database** stores data in **tables**. A table looks like a spreadsheet: **columns** are the kinds of information, and **rows** are the records (one row = one student).

Your database is called `crud_operations` and it has one table called `students`:

Column	Type	What it means
<code>id</code>	<code>int, AUTO_INCREMENT, PRIMARY KEY</code>	A unique number for each student. MySQL fills it in automatically (1, 2, 3...). No two students can have the same id, so it's how the app knows which student to update or delete.
<code>firstName</code>	<code>varchar(255)</code>	Text, up to 255 characters.
<code>lastName</code>	<code>varchar(255)</code>	Text, up to 255 characters.
<code>age</code>	<code>int(3)</code>	A whole number.

Here's what the table might look like with some data in it:

<code>id</code>	<code>firstName</code>	<code>lastName</code>	<code>age</code>
1	Sara	Ahmed	20
2	Omar	Ali	22
5	Lina	Saleh	19

Why is there no 3 or 4?

If you delete students 3 and 4, their numbers are **not** reused. The next student gets the next new number. That's normal, and it means an id always points to one student.

4. The files, and how a page is built

File	Job
<code>index.php</code>	The home page. Shows the table of students and the ADD STUDENT pop-up.
<code>insert_data.php</code>	Saves a new student (Create).
<code>update.php</code>	Shows a student's data in a form and saves changes (Update).
<code>delete.php</code>	Removes a student (Delete).
<code>dbconnection.php</code>	Opens the connection to MySQL.
<code>header.php</code>	The top of every page (title bar, Bootstrap, CSS).
<code>footer.php</code>	The bottom of every page (closes tags, loads Bootstrap JavaScript).
<code>style.css</code>	Your own design rules (colors, spacing, icon size).

Every page is built like a **sandwich**. Instead of writing the same top and bottom in every file, you write them once and pull them in with `include`:

1 `header.php`: top of the page (the brown title bar)



2 The page's own content (for example the student table in `index.php`)



3 `footer.php`: bottom of the page

`index.php` (simplified)

```
<?php include('header.php'); ?>
<?php include('dbconnection.php'); ?>

... the page's own content ...

<?php include('footer.php'); ?>
```

`include('header.php')` means: **"paste the code from header.php right here."** If you change the title in `header.php`, it changes on every page at once.

5. dbconnection.php: talking to the database

dbconnection.php

```
<?php

define("HOSTNAME","localhost");
define("USERNAME","root");
define("PASSWORD","");
define("DATABASE","crud_operations");

$connection = mysqli_connect(HOSTNAME, USERNAME, PASSWORD, DATABASE);

if (!$connection){
    die("Connection Failed");
}
```

Code	What it means
<code>define("HOSTNAME", ...)</code>	Creates a constant : a name for a value that never changes. <code>HOSTNAME</code> now means <code>"localhost"</code> .
<code>USERNAME / PASSWORD</code>	The MySQL login. XAMPP's default is user <code>root</code> with an empty password.
<code>mysqli_connect(...)</code>	Opens a connection to MySQL using those four values.
<code>\$connection = ...</code>	Saves the connection in a variable called <code>\$connection</code> . In PHP, every variable starts with <code>\$</code> .
<code>if (!\$connection)</code>	<code>!</code> means "not". So: "if the connection did not work..."
<code>die("Connection Failed")</code>	Stops the whole page and shows this message.

Think of \$connection as a phone line

`mysqli_connect` dials MySQL. `$connection` is the open line. Every time you want to ask MySQL something, you use that line: `mysqli_query($connection, ...)`.

Remember the error "Undefined variable \$connection"?

`insert_data.php` used `$connection` but never included `dbconnection.php`, so the variable didn't exist there.

Every PHP file runs on its own. A variable created in one file only exists in another file if you `include` it. That's why every file that talks to the database starts with `include('dbconnection.php');`.

6. header.php and footer.php

header.php (Bootstrap link shortened)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Home Page</title>
  <link href="../../../bootstrap.min.css" rel="stylesheet" ...>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1 id="main_title">
    <a href="index.php" class="icon"></a>
    CRUD APPLICATION IN PHP
  </h1>
  <br>
  <div class="container">
    <div class="wholePage">
```

Bootstrap is a free library of ready-made designs. Instead of writing CSS for every button and table, you add class names like `btn btn-success` and Bootstrap styles them for you. Your `style.css` is loaded **after** Bootstrap, so your own rules win if they disagree.

Notice the last two lines **open** two `<div>` tags but don't close them. They're closed in the footer:

footer.php

```
</div>
</div>
<script src="../../../bootstrap.bundle.min.js"></script>
</body>
</html>
```

Why does the footer load a JavaScript file?

Bootstrap has two parts: the **CSS** makes things look nice, and the **JavaScript** makes things move (like opening the ADD STUDENT pop-up). Without this `<script>` line, the button looks right but does nothing when you click it. That was the first bug you hit.

7. Read: showing the students

This is the **R** in CRUD. In `index.php`, PHP asks MySQL for every student, then makes one table row for each one.

`index.php`

```
<?php
$query = "SELECT * FROM `students`";
$result = mysqli_query($connection, $query);
if (!$result) {
    die("query Failed");
} else {
    while ($row = mysqli_fetch_assoc($result)) {
        ?>
        <tr>
            <td><?php echo $row['id']; ?></td>
            <td><?php echo $row['firstName']; ?></td>
            <td><?php echo $row['lastName']; ?></td>
            <td><?php echo $row['age']; ?></td>
            <td><a href="update.php?id=<?php echo $row['id']; ?>" class="btn
                btn-success">Update</a></td>
            <td><a href="delete.php?id=<?php echo $row['id']; ?>" class="btn
                btn-danger">Delete</a></td>
        </tr>
    <?php
    }
}
?>
```

Step by step

Code	What it means
<code>SELECT * FROM students</code>	SQL for: "give me all columns (*) of all rows in the students table."
<code>mysqli_query(\$connection, \$query)</code>	Sends the SQL to MySQL through the connection.
<code>\$result</code>	Holds all the rows MySQL sent back, like a stack of cards.
<code>while (...)</code>	A loop : repeat the code inside again and again, as long as the condition is true.
<code>mysqli_fetch_assoc(\$result)</code>	Takes the next card off the stack. When there are no cards left, it gives back nothing and the loop stops.
<code>\$row['firstName']</code>	One student's first name. <code>\$row</code> is an array (see below).
<code>echo</code>	Prints something into the HTML page.

Each time through the loop, `$row` holds one student as an **array**, a list of labelled values:

What `$row` looks like for one student

```
$row = [
    'id'          => 1,
    'firstName'   => 'Sara',
    'lastName'    => 'Ahmed',
    'age'         => 20
];
```

Jumping between PHP and HTML

`?>` means "stop PHP, the next lines are plain HTML." `<?php` means "start PHP again."

That's how the `<tr>` row (HTML) can sit **inside** the `while` loop (PHP). The loop repeats the HTML once for each student, so 3 students give 3 rows.

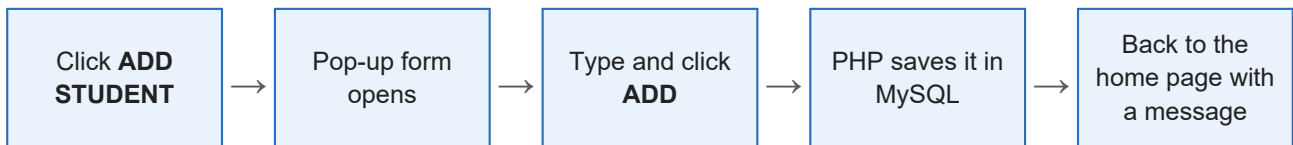
How Update and Delete know which student

Look at the link: `update.php?id=<?php echo $row['id']; ?>`. For the student with id 5, PHP prints `update.php?id=5`.

The part after `?` is called the **query string**. It carries the id to the next page, which reads it with `$_GET['id']`.

8. Create: adding a student

Adding a student takes two files: the **form** in `index.php` collects the data, and `insert_data.php` saves it.



Part 1: the button and the pop-up (index.php)

The button

```
<button type="button" class="btn btn-primary"
        data-bs-toggle="modal" data-bs-target="#exampleModal">ADD STUDENT</button>
```

A pop-up window is called a **modal**. `data-bs-toggle="modal"` tells Bootstrap "this button opens a modal", and `data-bs-target="#exampleModal"` says **which** one: the element with `id="exampleModal"`.

The form (shortened)

```
<form action="insert_data.php" method="post">
  <div class="modal fade" id="exampleModal" ...>
    ...
    <div class="mb-3">
      <label for="firstName" class="form-label">First Name</label>
      <input type="text" name="firstName" id="firstName" class="form-control">
    </div>
    ... (Last Name and Age are the same) ...
    <input type="submit" class="btn btn-success" name="addStudent" value="ADD">
  </div>
</form>
```

Code	What it means
<code>action="insert_data.php"</code>	Where to send the data when the form is submitted.
<code>method="post"</code>	How to send it: hidden inside the request, not in the address bar.
<code>name="firstName"</code>	The label the data travels with. In PHP, what the user typed arrives as <code>\$_POST['firstName']</code> .
<code>type="submit"</code>	Clicking this button sends the form.
<code>name="addStudent"</code>	Lets PHP check that the form was really submitted (see below).
<code>mb-3, form-label, form-control</code>	Bootstrap classes for spacing and nice-looking inputs.

Part 2: saving it (insert_data.php)

insert_data.php

```
<?php
include('dbconnection.php');

if (isset($_POST["addStudent"])) {

    $firstName=$_POST["firstName"];
    $lastName=$_POST["lastName"];
    $age=$_POST["age"];

    if($firstName == '' || empty($firstName)){
        header('location:index.php?message=You need to fill the first name!');
        exit;
    }
    else{
        $query = "insert into `students` (`firstName`,`lastName`,`age`) values
            ('$firstName','$lastName','$age')";
        $result = mysqli_query($connection,$query);

        if(!$result){
            die("Query Failed".mysqli_error($connection));
        }
        else{
            header('location:index.php?insert_msg=Your data has been added successfully!');
            exit;
        }
    }
}
```

Code	What it means
<code>isset(\$_POST["addStudent"])</code>	"Was the ADD button clicked?" isset checks if something exists. If someone opens this file directly, nothing happens.
<code>\$firstName=\$_POST["firstName"]</code>	Copies what the user typed into a simple variable.
<code>\$firstName == ''</code>	"Is the first name empty?" == compares, means "or".
<code>header('location:...')</code>	Redirect: tells the browser "go to this other page now."
<code>exit;</code>	Stop running this file. The redirect is sent, there's nothing else to do.
<code>insert into ... values (...)</code>	SQL for: "add a new row with these values." We don't give an id , MySQL creates it.
<code>mysqli_error(\$connection)</code>	If the query failed, this explains why . Very useful for debugging.

Why does it redirect instead of showing a page?

`insert_data.php` has no HTML of its own. Its only job is to save the data. Then it sends you back to `index.php`, where you see the updated table with the new student in it.

9. Messages after each action

When `insert_data.php` redirects to `index.php?insert_msg=Your data has been added successfully!`, the message travels **in the address**. `index.php` reads it with `$_GET`:

`index.php`

```
<?php
if(isset($_GET['insert_msg'])) {
    echo "<h5>".$_GET['insert_msg']."</h5>";
}
?>
```

In plain words: **"If the address contains `insert_msg`, print its text inside an `<h5>`."** The dot `.` joins pieces of text together.

There are four messages. Each uses its own name, so the page knows which kind it is:

Name in the address	Sent by	Shown in	Color
<code>message</code>	<code>insert_data.php</code> (first name empty)	<code><h6></code>	Red
<code>insert_msg</code>	<code>insert_data.php</code> (success)	<code><h5></code>	Green
<code>update_msg</code>	<code>update.php</code> (success)	<code><h5></code>	Green
<code>delete_msg</code>	<code>delete.php</code> (success)	<code><h6></code>	Red

The colors come from `style.css`. The rule `h6 { color: red; }` colors **every** `<h6>` red, and `h5 { color: green; }` colors every `<h5>` green:

`style.css`

```
h6{
    text-align: center;
    color: red;
}

h5{
    text-align: center;
    color: green;
}
```

Remember why the success message was red?

At first, both messages were sent as `message`, so both went into the red `<h6>`. CSS can't read the words in a message, it only sees the tag. Giving success its own name (`insert_msg`) sent it to the green `<h5>` instead.

10. Update: changing a student

`update.php` has **two jobs** in one file:

1 You click **Update** next to a student. The browser opens `update.php?id=5`.



2 **Job 1 (show):** PHP reads `$_GET['id']`, gets that student from MySQL, and fills the form with their data.



3 You change something and click **UPDATE**. The form is sent (POST) to `update.php?id_new=5`.



4 **Job 2 (save):** PHP runs an `UPDATE` query with the new values.



5 PHP redirects you to `index.php?update_msg=...`

Job 1: show the student in the form

update.php

```
if (isset($_GET['id'])){
    $id = (int) $_GET['id'];

    $query = "SELECT * FROM `students` where `id` = '$id'";
    $result = mysqli_query($connection, $query);
    if (!$result) {
        die("query Failed");
    } else {
        $row= mysqli_fetch_array($result);
    }
}
```

where `id = '$id'` means "only the student with this id." `(int)` turns the value into a whole number, so nobody can sneak SQL code in through the address bar.

update.php: the form

```
<form action="update.php?id_new=<?php echo $id; ?>" method="post">
    <input type="text" name="firstName" ... value="<?php echo
        htmlspecialchars($row['firstName']); ?>">
    ...
    <input type="submit" class="btn btn-success" name="update" value="UPDATE">
</form>
```

`value="..."` puts text **inside** the input before the user types anything. That's how the form starts filled with the current name. `htmlspecialchars` makes the text safe to put inside HTML.

Job 2: save the changes

update.php

```
if (isset($_POST['update'])){
    if(isset($_GET['id_new'])){
        $idnew = (int) $_GET['id_new'];
    }

    $firstName= $_POST['firstName'];
    $lastName= $_POST['lastName'];
    $age= $_POST['age'];

    $query="update `students` set `firstName` = '$firstName', `lastName` = '$lastName',
        `age` = '$age' where `id`='$idnew'";
    $result = mysqli_query($connection, $query);
    if (!$result) {
        die("query Failed".mysqli_error($connection));
    } else {
        header('location:index.php?update_msg=You have successfully updated the data.');
```

`update ... set ... where id = ...` is SQL for: "in the row with this id, change these columns to these values."

Never forget WHERE

Without `WHERE id = ...`, the `UPDATE` changes **every** student in the table to the same name. The same goes for `DELETE`: without `WHERE`, it deletes everyone.

Why is the save code at the very top of the file?

`header('location:...')` only works **before** any HTML has been sent to the browser.

`include('header.php')` sends HTML (the title bar), so the redirect must happen before it.

Think of it like mailing a letter: you have to write the address on the envelope **before** you post it. Once the letter (the HTML) is on its way, it's too late to change where it goes. If you get it wrong, PHP shows: **"Cannot modify header information - headers already sent"**.

The bug that stopped Update from saving

The query said `where id='id_new'`, with quotes but no `$`. MySQL looked for a student whose id is the **word** "id_new". None exists, so nothing changed and there was no error.

Rule: `'id_new'` in quotes is just text. `$idnew` is a variable holding a value.

11. Delete: removing a student

delete.php (as it is now)

```
<?php include('header.php'); ?>
<?php include('dbconnection.php'); ?>

<?php
    if(isset($_GET['id'])){
        $id = $_GET['id'];

        $query = "delete from `students` where `id` = '$id'";
        $result= mysqli_query($connection,$query);
        if(!$result){
            die("Query Failed".mysqli_error($connection));
        } else{
            header('location:index.php?delete_msg=You have deleted the record.');
```

The idea is simple: read the id from the address, run `delete from students where id = ...`, then go back to `index.php`.

This file still has a bug you now know how to spot

It includes `header.php` (which sends HTML) **before** calling `header('location:...')`. The student is deleted, but the redirect fails and you're left on an empty page with the title bar.

Fix: this page never shows anything, so it doesn't need `header.php` or `footer.php` at all:

delete.php (fixed version)

```
<?php
include('dbconnection.php');

if(isset($_GET['id'])){
    $id = (int) $_GET['id'];

    $query = "delete from `students` where `id` = '$id'";
    $result= mysqli_query($connection,$query);
    if(!$result){
        die("Query Failed".mysqli_error($connection));
    } else{
        header('location:index.php?delete_msg=You have deleted the record.');
```

12. GET vs POST

There are two ways to send data from the browser to PHP. Your app uses both.

	GET	POST
Where is the data?	In the address: <code>delete.php?id=5</code>	Hidden inside the request
Read it in PHP with	<code>\$_GET['id']</code>	<code>\$_POST['firstName']</code>
Visible to the user?	Yes, in the address bar	No
Used in your app for	Update/Delete links (<code>?id=5</code>), messages (<code>?insert_msg=...</code>)	The ADD form and the UPDATE form
Good for	Small things like an id or a page number	Forms, passwords, longer data

13. The errors you hit, and what they teach

You fixed a lot of bugs building this app. Every developer hits these same ones. Here's each one, its cause, and the lesson to keep:

What happened	Cause	Lesson
ADD STUDENT button did nothing	The code used Bootstrap 4 attributes (<code>data-toggle</code>) but the page loads Bootstrap 5. Also, Bootstrap's JavaScript file wasn't loaded.	Check which version a tutorial uses. Bootstrap 5 uses <code>data-bs-....</code> . Pop-ups need the JS file.
No space between label and input	<code>form-group</code> was removed in Bootstrap 5.	Use <code>mb-3</code> and <code>form-label</code> in Bootstrap 5.
New CSS rule didn't show	The browser was using an old saved (cached) copy of <code>style.css</code> .	Press Ctrl + F5 to force a full reload.
"Undefined variable \$connection"	<code>insert_data.php</code> didn't include <code>dbconnection.php</code> .	Every PHP file runs on its own. Include what you need.
Success message was red	Success and error used the same name and the same <code><h6></code> tag.	Give different things different names or classes.
Update/Delete buttons not styled	Classes were written outside <code>class="..."</code> .	Class names always go inside <code>class="..."</code> .
Update page crashed	An <code>if {</code> was never closed with <code>}</code> .	Every <code>{</code> needs a matching <code>}</code> . VS Code highlights the pair.
Form was empty / data not saved	The query used <code>'id'</code> and <code>'id_new'</code> (text) instead of <code>\$id</code> and <code>\$idnew</code> .	Variables start with <code>\$</code> . Quotes alone make plain text.
"Headers already sent"	<code>header('location:...')</code> ran after <code>header.php</code> had sent HTML.	Redirect before any HTML is output.
Icon was huge	The CSS sized the <code><a></code> link, not the <code></code> inside it.	Target the right element: <code>.icon img</code> .

A good habit for debugging

1. **Read the error message.** It tells you the file and the line number.
2. **Check the spelling** of variable names, column names and class names. One letter matters.
3. **Use `mysqli_error($connection)`** after a query to see what MySQL didn't like.
4. **Change one thing at a time**, then refresh and test.

14. Next steps

Your app works, but it isn't safe to put on the real internet yet. These are the next things to learn, in order:

Topic	Why it matters
Prepared statements (<code>mysqli_prepare</code>)	Right now, what the user types goes straight into your SQL. A name like <code>O'Brien</code> breaks the query, and an attacker could type SQL code to change or erase your data. This is called SQL injection . Prepared statements fix it.
<code>htmlspecialchars()</code> on all output	<code>index.php</code> prints messages and names exactly as they are. Someone could put HTML or JavaScript in them and it would run on your page.
Confirm before deleting	One click deletes a student forever. Add a "Are you sure?" step.
Validate all fields	Only the first name is checked. Age accepts any text, like "abc".

Here's what a prepared statement looks like for the insert, so you can recognise it later:

Safer insert

```
$stmt = mysqli_prepare($connection,
    "INSERT INTO `students` (`firstName`,`lastName`,`age`) VALUES (?, ?, ?)");
mysqli_stmt_bind_param($stmt, "ssi", $firstName, $lastName, $age);
$result = mysqli_stmt_execute($stmt);
```

The `?` marks are empty slots. "`ssi`" means the three values are **s**tring, **s**tring, **i**nteger. MySQL fills the slots safely, so quotes in a name can't break anything.

15. Glossary

Word	Meaning
Variable	A named box that holds a value. In PHP it starts with <code>\$</code> , like <code>\$firstName</code> .
Constant	Like a variable, but its value can't change. Made with <code>define()</code> .
Array	A list of values with labels, like <code>\$row['firstName']</code> .
Database	A program that stores data permanently in tables (MySQL).
Table / row / column	A table is like a spreadsheet. A row is one record (one student), a column is one kind of information (age).
SQL	The language for talking to databases: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> .
Query	One SQL command sent to the database.
Primary key	A column whose value is unique for every row (<code>id</code>).
include	Pastes another PHP file's code into the current file.
\$_GET	Data that came in the address, after the <code>?</code> .
\$_POST	Data that came from a form with <code>method="post"</code> .
isset()	Checks whether something exists.
echo	Prints text into the page.
Redirect	Telling the browser to go to another page: <code>header('location:...')</code> .
Loop (<code>while</code>)	Repeats code as long as a condition is true.
Bootstrap	A library of ready-made designs you use through class names.
Modal	A pop-up window on top of the page.
Cache	Saved copies of files the browser keeps to load pages faster.
localhost	Your own computer, acting as the server.